

Assistant for a Better You

Abstract

The main motivation for *Assistant for a Better You* was to create a smart assistant that helps a user keep track of their habits in an interactive fashion. The user can tell the assistant what habits they would like to add to their life. The device would help keep a user accountable by reminding them of these goals and asking the user to report whenever they complete one of the goals. A gamified tally system, potentially causing the human user to feel accountable to the device and, by proxy, to their own habits.

The system was created using an ultrasonic motion detector, an I2S microphone module, and an I2S amplifier and 3W speaker. The motion detector is used to wake the device up, the microphone is used for the user to give commands, and the speaker is the device's form of feedback. A remote server running a python program handles the speech-to-text inference and text-to-speech generation. The device and the server communicate via WiFi. The ESP32 receives transcripts of the user's speech and finds a matching command to allow the user to control it. Some files are also preloaded or saved to the flash memory for improved quality of use.

The results of the project were successful overall. Once the modules were configured and the finite state machine was organized, the major functionality was easily handled. There is still a lot of room for improvement, especially when it comes to security, efficiency, and functionality. However, almost all major functions were successfully implemented. Whether the device is effective at helping users with their habits requires additional observation, but overall, the main objectives were completed.

Motivation and Background

I've struggled with building consistent habits in the past, especially when it comes to activities that are good for me but are not necessarily the most fun. One method that was effective was turning the changes into a game. I'd create a short checklist of things I want to spend a small amount of time on daily to build up preferred habits. After spending some time on a habit, I'd go add a tally mark to that item to indicate that I finished it. This helped me feel good when I collected a lot of tally marks, indicating I was succeeding at my habit changes. However, I found that it's easy to simply ignore the list when times get stressful. I want to create a device that speaks to me about my list when I approach my mirror, and that allows me to report back to it to mark off items as "complete".

Project Goals

The goal of my project was to create a smart assistant device that helps the user maintain their goals. A user can tell the device the daily habits they wish to change, and the device can remember their list and remind the user at a later time. It detects if a user is nearby via a motion sensor and prompts them for a command. The device also tracks the tally of the habits that have been completed. It has persistent data storage to maintain the habits and tallies after losing power or rebooting. The main goal is to make something that's convenient to use, but more human, engaging, and interactive than a simple pad and pen used to track habits.

Assumptions

The main assumption I made for this project is that this is a simple demo of a fully functioning commercial device. There are so many additional features I would need to add or improve, as well as security threats that I would need to resolve if I were to try to give this to a real user.

Another assumption is that the user would know how to control or set up the device without instructions, as there is no help function and no way to set up the Wi-Fi without having access to the ESP32 and python code. In other words, the major assumption is that this is a simple proof of concept to test if this device would be helpful.

Other major assumptions that are important relate to accessibility. The device only works for people who can understand or speak English, or who are not deaf or mute. Lastly, a major assumption is that the user would want help improving their habits by having an assistant keep them accountable, which may not be true of everyone trying to change their habits.

System Architecture

Sensors

Ultrasonic sensor: This sensor detects the distance of the nearest object. Changes in this distance can be used to detect nearby motion that will wake up the device.

I2S microphone: This sensor sends sound data to the ESP32, which then can stream the audio over WiFi via UDP. This can be used to interpret commands from the user.

Actuator

I2S Amplifier + 3W speaker: This combination of the amp module and speaker allows the ESP32 to play .mp3 files using the Arduino Audio library.

File System

SPIFFS: This acts as the onboard memory for the ESP32, allowing common response phrases that the assistant uses to be preloaded onto the device. It's also used to store a JSON file of past habits data so that it persists after the device loses and regains power.

Personal Computer

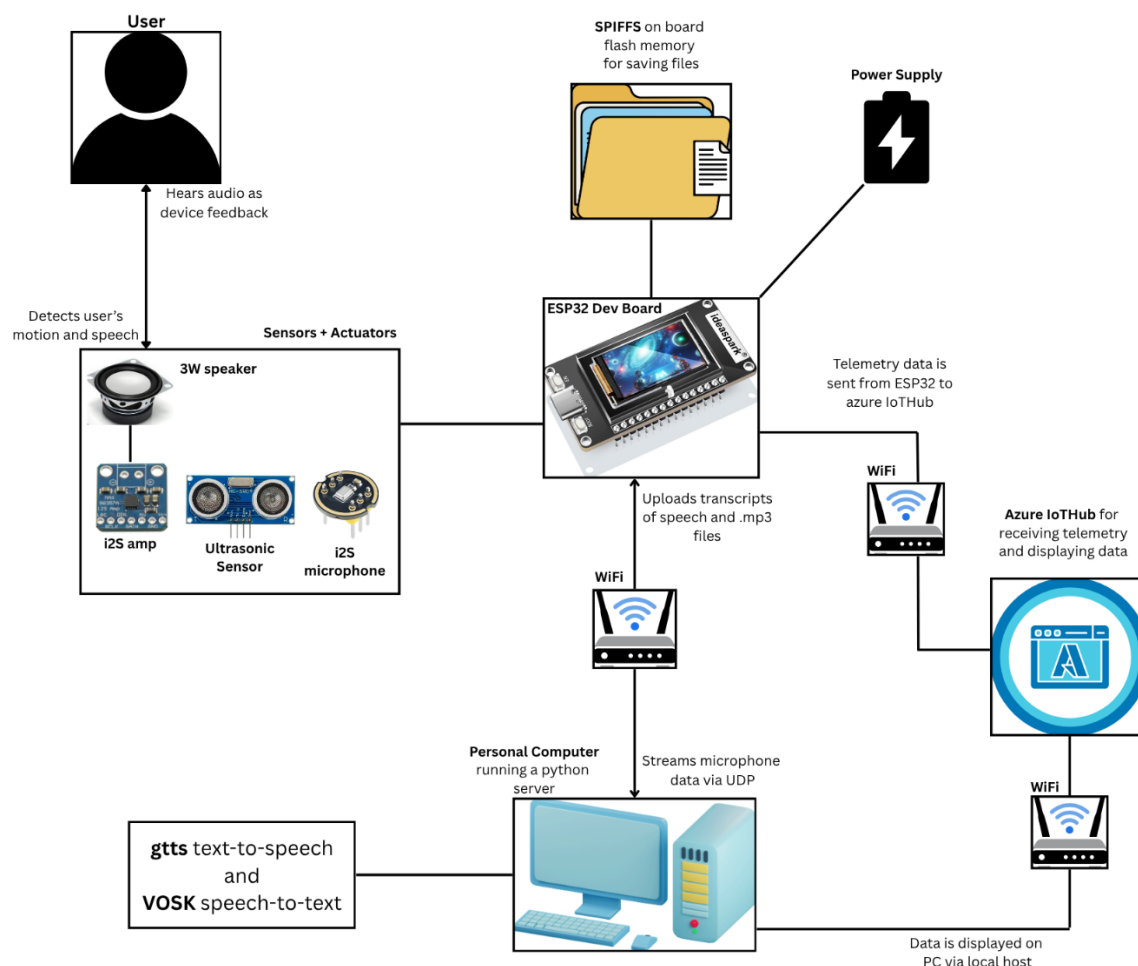
A personal computer was used to host a python server running a speech-to-text library and a text-to-speech generation library. It also receives data from the IoT Hub to visualize via a local host program.

Azure IoT Hub

The IoT Hub receives telemetry data from the ESP32 via WiFi in the form of JSON files.

Libraries

Many libraries were used, including many Arduino libraries, such as WiFi, WiFiClient, WiFiUDP, HTTPClient, ArduinoJSON, WebServer, Audio, SPIFFS, as well as the I2S driver. Many example programs were also adapted to fit the goals and architecture of the device.



Components

1. ideaspark ESP32 Development Board
2. INMP441 Omnidirectional Microphone Module
3. Gikfun 2" 4Ohm 3W Full Range Audio Speaker Stereo Woofer Loudspeaker for Arduino EK1725
4. MAX98357 MAX98357A I2S 3W Class D Amplifier Breakout Interface I2S
5. WWZMDiB HC-SR04 Ultrasonic Sensor Module

Tasks

- Configured, soldered, and wired the sensors and actuators, including an i2s amplifier and speaker for playing mp3 files, a microphone for streaming audio, and an ultrasonic motion sensor for waking up the device.
- Researched the necessary libraries and adapted example sketches to fit the project and complete its goals.
- Created functionality that stores a persistent JSON file to allow the device to continue remembering the user's habits data after rebooting.
- Creatively planned and engineered a finite state machine architecture that fits the project's goals, software, libraries, and features.
- Configured multiple servers, a python server on a personal computer, an HTTP server on the ESP32, and an Azure cloud server that receives telemetry to visualize data.

Challenges

I faced many different roadblocks. One was figuring out how to upload mp3 files to the device itself. I tried uploading mp3 files through UDP, but it proved difficult because the mp3 file would be cut short. I thought it might've been because data was arriving out of order, so I tried to figure out if there was a TCP protocol I could use, but I wasn't able to find anything that I could figure out in time. Ultimately, I decided to run an HTTP server on the ESP32 itself and have the python server upload the mp3 file to the HTTP server. It was a much slower approach in terms of upload speed, but it worked consistently.

Another roadblock I faced was trying to use the MQTT protocol coupled with Azure IoT Hub device twins feature to store the habits data so it would persist after the device lost power. After many hours of debugging, I never figured out the MQTT protocol. However, I salvaged the functions that created/interpreted the JSON file containing the habit data by having the ESP32 save the JSON file to its own flash and then read the JSON file on boot up. I also used the JSON file to send telemetry data to the Azure cloud for data visualization.

Future Work

More humanlike interaction - A major point of my project was to make the device more compelling than a pad labeled “daily habits” sitting on your desk. While it has mostly succeeded in this effort, more could’ve been done to make it feel alive.

Functionality and user experience - There are still many functions that need to be added and current functions that need to be tested further for a better user experience. The ability to delete a specific habit, yes or no confirmation on specific commands, a “help” function to list user commands, or a cancel function to stop a particular process would be necessary for a better user experience. The ability to have the tally spoken back to the user is also pivotal to the main goal of the project.

Error detection, security, efficiency - There are many parts of the program that are error prone. Errors should try to resolve themselves when failing to connect, or at the very least offer the user better feedback. Habit data can be scraped by malicious actors trying to build an identity profile on a user. Security needs to be accounted for. Efficiency is also important. File transfer can be slow. The code inside the ESP32 can also be improved by looking at what libraries/variables can be removed/consolidated, or where the program execution could be quicker.

Demo Video

<https://drive.google.com/file/d/1DxG2uaHQa4dvGQXTb-kd0v01mmjGa7P1/view?usp=sharing>